

Contents

Project 1: Satellite Image Data Analysis using NumPy.....	1
Project 2: Visualization of world GDP and carbon - dioxide emission	5
Project 3: Visualization: Using Folium Library for Geographic Overlays.....	9
Project 4: Sentiment Analysis using Logistic Regression	10
Project 5: Daily Weather Data Analysis using Decision Tree Classification	15
Project 6: Nearest Neighbour for Handwritten Digit Recognition	18
Project 7:.....	23
Project 8:.....	26

PROJECT 1: SATELLITE IMAGE DATA ANALYSIS USING NUMPY

Data Source: Satellite Image from WIFIRE Project

WIFIRE is an integrated system for wildfire analysis, with specific regard to changing urban dynamics and climate. The system integrates networked observations such as heterogeneous satellite data and real-time remote sensor data, with computational techniques in signal processing, visualization, modeling, and data assimilation to provide a scalable method to monitor such phenomena as weather patterns that can help predict a wildfire's rate of spread.

RGB Color Mapping in the Photo:

- RED pixel indicates Altitude
- BLUE pixel indicates Aspect
- GREEN pixel indicates Slope

The higher values denote higher altitude, aspect and slope.

In this example, we will analyze a sample satellite image dataset from WIFIRE using the numpy Library.

```
#Loading the libraries we need: numpy, scipy, matplotlib
import numpy as np
import scipy
import imageio
import matplotlib.pyplot as plt
import skimage #install scikit-image
import warnings
warnings.filterwarnings("ignore")
```

```

#Creating a numpy array from an image file:
#Lets choose a WIFIRE satellite image file as an ndarray and display
its type
from skimage import data
photo_data = imageio.imread('data/sd-3layers.jpg')
type(photo_data)

#Let us see what is in this image.
plt.figure(figsize=(15,15))
plt.imshow(photo_data)
plt.show()
#Let us see how does the photo_data looks like:
print(photo_data)
#Let us print the shape of photo_data
print(photo_data.shape)
#The shape of the ndarray show that it is a three layered matrix.
# The first two numbers here are length and width, and
# the third number (i.e. 3) is for three layers: Red, Green and Blue.

#RGB Color Mapping in the Photo:
#RED pixel indicates Altitude
#BLUE pixel indicates Aspect
#GREEN pixel indicates Slope
#The higher values denote higher altitude, aspect and slope.
photo_data.size # Data Size
photo_data.min(), photo_data.max() # Maximum and Minimum of the pixel
value
photo_data.mean() # Average Pixel Value

#Pixel on the 150th Row and 250th Column:
photo_data[150, 250] # It will print the Red, Green , Blue Value in
order
photo_data[150, 250, 1] # It will print out the Green Value

#Pixel on the 2nd Row and 2nd Column:
photo_data[1, 1] # It will print the Red, Green, Blue Value in order
#Set a Pixel to All Zeros
#We can set all three layer in a pixel as once by assigning zero
globally to that
# (row,column) pairing. However, setting one pixel to zero is not
noticeable.
photo_data = imageio.imread('data/sd-3layers.jpg')
photo_data[150, 250] = 0 # We set all three layers of RGB of this
plt.figure(figsize=(10,10))
plt.imshow(photo_data)
plt.show()

#Changing colors in a Range
#We can also use a range to change the pixel values. As an example,
# let us set the green layer for rows 200 to 800 to full intensity.
#We will set the value of Green layer to full intensity for rows 200

```

```

(inclusive)
# to 800 (exclusive) for all the columns.
photo_data = imageio.imread('data/sd-3layers.jpg')
photo_data[200:800, :, 1] = 255
plt.figure(figsize=(10,10))
plt.imshow(photo_data)
plt.show()

#We will set the value of Red, Green and Blue layer to full intensity
# (we will get a white block) for rows 200 (inclusive) to 800
(exclusive) for all the columns.
photo_data = imageio.imread('data/sd-3layers.jpg')
photo_data[200:800, :] = 255
plt.figure(figsize=(10,10))
plt.imshow(photo_data)
plt.show()

#We will set the value of Red, Green and Blue layer to least intensity
# (we will get a black block) for rows 200 (inclusive) to 800
(exclusive) for all the columns.
photo_data = imageio.imread('data/sd-3layers.jpg')
photo_data[200:800, :] = 0
plt.figure(figsize=(10,10))
plt.imshow(photo_data)
plt.show()

#Pick all Pixels with Low Values
photo_data = imageio.imread('data/sd-3layers.jpg')
print("Shape of photo_data:", photo_data.shape)
low_value_filter = photo_data < 100 #Create a boolean array with the
same shape as of photo_data
print("Shape of low_value_filter:", low_value_filter.shape)

#Filtering Out Low Values:
#Whenever the low_value_filter is True, set value to 0.
plt.figure(figsize=(10,10))
plt.imshow(photo_data)
photo_data[low_value_filter] = 0 #set low values to 0
plt.figure(figsize=(10,10))
plt.imshow(photo_data)
plt.show()

#More Row and Column Operations
#We can design complex patterns by making columns a function
# of rows or vice-versa. Here we try a linear relationship between
rows and columns.
rows_range = np.arange(len(photo_data)) #Create a range array
print(rows_range)
cols_range = rows_range #Create a range array
print(cols_range)
print(type(rows_range))

```

```

#We are setting the selected rows and columns to the maximum value of
255
photo_data[rows_range, cols_range] = 255
print(photo_data)
plt.figure(figsize=(15,15))
plt.imshow(photo_data)
plt.show()
#We see a diagonal white line that is a result of our operation.

#Masking Images
#Now let us try to mask the image in shape of a circular disc.
total_rows, total_cols, total_layers = photo_data.shape
print("photo_data = ", photo_data.shape)
X, Y = np.ogrid[:total_rows, :total_cols]
print("X = ", X.shape, " and Y = ", Y.shape)
from IPython.display import Image
Image("Images/figure.png")
center_row, center_col = total_rows / 2, total_cols / 2
print("center_row = ", center_row, "AND center_col = ", center_col)
#print(X - center_row)
#print(Y - center_col)
dist_from_center = (X - center_row)**2 + (Y - center_col)**2
#print(dist_from_center)
radius = (total_rows / 2)**2
#print("Radius = ", radius)
circular_mask = (dist_from_center > radius)
#print(circular_mask)
print(circular_mask[1500:1700,2000:2200])
photo_data = imageio.imread('data/sd-3layers.jpg')
photo_data[circular_mask] = 0
plt.figure(figsize=(15,15))
plt.imshow(photo_data)
#Further Masking
#We can further improve the mask, for example just get upper half
disc.
X, Y = np.ogrid[:total_rows, :total_cols]
half_upper = X < center_row # this line generates a mask for all rows
above the center
half_upper_mask = np.logical_and(half_upper, circular_mask)
photo_data = imageio.imread('data/sd-3layers.jpg')
photo_data[half_upper_mask] = 255
#photo_data[half_upper_mask] = random.randint(200,255)
plt.figure(figsize=(15,15))
plt.imshow(photo_data)
plt.show()

```

Further Processing of our Satellite Imagery

* Processing of RED Pixels

* Remember that red pixels tell us about the height. Let us try to highlight all the high altitude areas. We will do this by detecting high intensity RED Pixels and muting down other areas.

```
photo_data = imageio.imread('data/sd-3layers.jpg')
red_mask = photo_data[:, :, 0] < 150
photo_data[red_mask] = 0
plt.figure(figsize=(15,15))
plt.imshow(photo_data)
plt.show()

#Detecting Highl-GREEN Pixels
photo_data = imageio.imread('data/sd-3layers.jpg')
green_mask = photo_data[:, :, 1] < 150
photo_data[green_mask] = 0
plt.figure(figsize=(15,15))
plt.imshow(photo_data)
plt.show()

#Detecting Highly-BLUE Pixels
photo_data = imageio.imread('data/sd-3layers.jpg')
blue_mask = photo_data[:, :, 2] < 150
photo_data[blue_mask] = 0
plt.figure(figsize=(15,15))
plt.imshow(photo_data)
plt.show()

#Composite mask that takes thresholds on all three layers: RED, GREEN, BLUE
photo_data = imageio.imread('data/sd-3layers.jpg')
red_mask = photo_data[:, :, 0] < 150
green_mask = photo_data[:, :, 1] > 100
blue_mask = photo_data[:, :, 2] < 100
final_mask = np.logical_and(red_mask, green_mask, blue_mask)
photo_data[final_mask] = 0
plt.figure(figsize=(15,15))
plt.imshow(photo_data)
plt.show()
```

PROJECT 2: VISUALIZATION OF WORLD GDP AND CARBON - DIOXIDE EMISSION

World Development Indicators

The World Development Indicators dataset obtained from the World Bank containing over a thousand annual indicators of economic development from hundreds of countries around the world.

```
# Initial exploration of the Dataset
```

```

import pandas as pd
import numpy as np
import random
import matplotlib.pyplot as plt
import matplotlib.cbook
import zipfile
import bz2
import warnings
warnings.filterwarnings("ignore",category=matplotlib.cbook.mplDeprecation)
#Let us read the dataset
data = pd.read_csv('data/Indicators.bz2')
#data = pd.read_csv('data/Indicators.bz2', compression = 'bz2')
print("data.shape: ", data.shape)
#This is a really large dataset, at least in terms of the number of rows.
print("Sample Data: \n",data.head())
print("Columns: \n",data.columns)

#From the above dataset, it looks like it has different indicators for
different
# countries with the year and value of the indicator.

#How many UNIQUE country names are there ?
countries = data['CountryName'].unique().tolist()
print("Number of countries: ",len(countries))
#Are there same number of country codes ?
#How many unique country codes are there?
#It should be the same as number of unique countries.
countryCodes = data['CountryCode'].unique().tolist()
print("Number of country codes: ",len(countryCodes))
#Are there many indicators or few ?
#How many unique indicators are there?
indicators = data['IndicatorName'].unique().tolist()
print("Number of indicators: ",len(indicators))
#How many years of data do we have ?
years = data['Year'].unique().tolist()
print("Number of years: ",len(years))
#What's the range of years?
print(min(years)," to ",max(years))

#####
## Data Visualization
#####
#Let us pick a country and an indicator to explore CO2 Emissions per capita
and the USA.
#To select CO2 emissions for the United States, We will take the intersection
# of two masks, one with all the rows that contains the string,
# "CO2 emissions" and the other which contains all the rows containing the
string, "USA".

hist_indicator = 'CO2 emissions \ (metric'
hist_country = 'USA'
mask1 = data['IndicatorName'].str.contains(hist_indicator)
mask2 = data['CountryCode'].str.contains(hist_country)
stage = data[mask1 & mask2]
# stage dataset contain indicators matching the USA for country code & CO2
emissions over time.
print (stage.shape)

```

```

stage.head()
print("Indicator Name: ", stage['IndicatorName'].iloc[0])

#Let us see how emissions have changed over time using Matplotlib
years = stage['Year'].values # get the years
co2 = stage['Value'].values # get the values
# Plot the Histogram
plt.bar(years,co2)
plt.show()

#It is seen that emissions per capita have dropped a bit over time,
# but let us make this graph a bit more appealing before we continue to
explore it.
#Let us create a line plot.
plt.plot(stage['Year'].values, stage['Value'].values)
# Label the axes
plt.xlabel('Year')
plt.ylabel(stage['IndicatorName'].iloc[0])
# Label the figure
plt.title('CO2 Emissions in USA')
# Start the y axis at 0 and x axis from 1959
plt.axis([1959, 2011,0,25])
plt.show()

#Using Histograms to explore the distribution of values
#We could also visualize this data as a histogram to better explore
# the ranges of values in CO2 production per year.
# If we want to just include those within one standard deviation fo the mean,
you could do the following
# lower = stage['Value'].mean() - stage['Value'].std()
# upper = stage['Value'].mean() + stage['Value'].std()
# hist_data = [x for x in stage[:10000]['Value'] if x>lower and x<upper ]
# Otherwise, let's look at all the data
hist_data = stage['Value'].values
print(hist_data)
print(len(hist_data))
# Histogram of the data
plt.hist(hist_data, 10, density=False, facecolor='green') # 10 is the number
of bins
plt.xlabel(stage['IndicatorName'].iloc[0])
plt.ylabel('# of Years')
plt.title('Histogram Example')
plt.grid(True)
plt.show()

#USA has many years where it produced between 19-20 metric tons per capita
with outliers on either side.
#But how do the USA's numbers relate to those of other countries?
# select CO2 emissions for all countries in 2011
hist_indicator = 'CO2 emissions \(metric'
hist_year = 2011
mask1 = data['IndicatorName'].str.contains(hist_indicator)
mask2 = data['Year'].isin([hist_year])
# apply our mask
co2_2011 = data[mask1 & mask2]
co2_2011.head()
#For how many countries do we have CO2 per capita emissions data in 2011

```

```

print(len(co2_2011))
# Let us plot a histogram of the emmissions per capita by country
# subplots returns a tuple with the figure, axis attributes.
fig, ax = plt.subplots()
ax.annotate("USA",xy=(18, 5), xycoords='data',xytext=(18, 30),
textcoords='data',
            arrowprops=dict(arrowstyle="->",connectionstyle="arc3"))
plt.hist(co2_2011['Value'], 10, density=False, facecolor='green')
plt.xlabel(stage['IndicatorName'].iloc[0])
plt.ylabel('# of Countries')
plt.title('Histogram of CO2 Emissions Per Capita')
plt.grid(True)
plt.show()

#USA, at ~18 CO2 emissions (metric tons per capital) is quite high among all
countries.
#3. Matplotlib: Basic Plotting Part 2
#Relationship between GPD and CO2 Emissions in USA
# Select GDP Per capita emissions for the United States
hist_indicator = 'GDP per capita \ (constant 2005'
hist_country = 'USA'
mask1 = data['IndicatorName'].str.contains(hist_indicator)
mask2 = data['CountryCode'].str.contains(hist_country)
# Stage is just those indicators matching the USA for country code and CO2
emissions over time.
gdp_stage = data[mask1 & mask2]
# Plot gdp_stage vs stage
print("GDP: ",gdp_stage.head())
stage.head()
# Switch to a line plot
plt.plot(gdp_stage['Year'].values, gdp_stage['Value'].values)
# Label the axes
plt.xlabel('Year')
plt.ylabel(gdp_stage['IndicatorName'].iloc[0])
#Label the figure
plt.title('GDP Per Capita USA')
plt.show()

#Although we have seen a decline in the CO2 emissions per capita,
# it does not seem to translate to a decline in GDP per capita

#ScatterPlot for comparing GDP against CO2 emissions (per capita)
#First, we will need to make sure we are looking at the same time frames.
print("GDP Min Year = ", gdp_stage['Year'].min(), "max: ",
gdp_stage['Year'].max())
print("CO2 Min Year = ", stage['Year'].min(), "max: ", stage['Year'].max())

#We have 3 extra years of GDP data, so let's trim those off so the
scatterplot
# has equal length arrays to compare (this is actually required by
scatterplot)
gdp_stage_trunc = gdp_stage[gdp_stage['Year'] < 2012]
print(len(gdp_stage_trunc))
print(len(stage))

import matplotlib.pyplot as plt
fig, axis = plt.subplots()

```

```

# Grid lines, Xticks, Xlabel, Ylabel
axis.yaxis.grid(True)
axis.set_title('CO2 Emissions vs. GDP (per capita)',fontsize=10)
axis.set_xlabel(gdp_stage_trunc['IndicatorName'].iloc[0],fontsize=10)
axis.set_ylabel(stage['IndicatorName'].iloc[0],fontsize=10)
X = gdp_stage_trunc['Value']
Y = stage['Value']
axis.scatter(X, Y)
plt.show()
#This does not look like a strong relationship. We can test this by looking at correlation.
print(np.corrcoef(gdp_stage_trunc['Value'],stage['Value']))

#A correlation of 0.07 is very weak.

```

PROJECT 3: VISUALIZATION: USING FOLIUM LIBRARY FOR GEOGRAPHIC OVERLAYS

Project 3 : Using Folium Library for Geographic Overlays

Further exploring CO2 Emissions per capita in the World Development Indicators Dataset

```

import folium
import pandas as pd
import warnings
warnings.filterwarnings("ignore")
country_geo = 'data/world-countries.json'
# Read in the World Development Indicators Database
data = pd.read_csv('data/Indicators.bz2', compression='bz2')
data.shape

print(data.head())

#Select CO2 emissions for all countries in 2011
hist_indicator = 'CO2 emissions \(metric'
hist_year = 2011
mask1 = data['IndicatorName'].str.contains(hist_indicator)
mask2 = data['Year'].isin([hist_year])
# Apply our mask
stage = data[mask1 & mask2]
stage.head()

#Setup the data for plotting
#Create a data frame with just the country codes and the values we want to be plotted.
plot_data = stage[['CountryCode','Value']]
plot_data.head()
# Label for the legend
hist_indicator = stage.iloc[0]['IndicatorName']
print(hist_indicator)

```

```

#Visualize CO2 emissions per capita using Folium
#Folium provides interactive maps with the ability to create sophisticated
# overlays for data visualization
#Setup a folium map at a high-level zoom.
map = folium.Map(location=[100, 0], zoom_start=1.5)

#Choropleth maps bind Pandas Data Frames and json geometries.
# This allows us to quickly visualize data combinations
map.choropleth(geo_data=country_geo, data=plot_data, columns=['CountryCode',
'Value'],
               key_on='feature.id', fill_color='YlGnBu', fill_opacity=0.7,
line_opacity=0.2,
               legend_name=hist_indicator)

# Create Folium plot
map.save('saved_info/plot_data.html')

# Import the Folium interactive html file
from IPython.display import HTML
HTML('<iframe src=saved_info/plot_data.html width=700 height=450></iframe>')

```

PROJECT 4: SENTIMENT ANALYSIS USING LOGISTIC REGRESSION

- The "sentiment" dataset consists of 3000 sentences which come from reviews on "imdb.com", "amazon.com", and "yelp.com". Each sentence is labeled according to whether it comes from a positive review or negative review.
- The data set consists of 3000 sentences, each labeled '1' (if it came from a positive review) or '0' (if it came from a negative review). To be consistent with our notation from lecture, we will change the negative review label to '-1'.
- We will use logistic regression to learn a classifier from this data.

1. Load and Preprocess data

```

import string
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
matplotlib.rc('xtick', labelsiz=14)
matplotlib.rc('ytick', labelsiz=14)
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import SGDClassifier
import warnings
warnings.filterwarnings("ignore")

#Read in the data set:
with open("data/full_set.txt") as f:
    content = f.readlines()

```

```

#Remove leading and trailing white space
content = [x.strip() for x in content]
#Separate the sentences from the labels:
sentences = [x.split("\t")[0] for x in content]
labels = [x.split("\t")[1] for x in content]
#Transform the labels from '0 versus 1' to '-1 versus 1':
y = np.array(labels, dtype='int8')
y = 2*y - 1

```

Preprocessing the text data

- To transform this prediction problem into an linear classification, we will need to preprocess the text data. We will do four transformations:
 - Remove punctuation and numbers.
 - Transform all words to lower-case.
 - Remove stop words.
 - Convert the sentences into vectors, using a bag-of-words representation.
- We begin with first two steps

```

#"full_remove" takes a string x and a list of characters
"removal_list" and returns with all the characters in removal_list
replaced by ' '.
def full_remove(x, removal_list):
    for w in removal_list:
        x = x.replace(w, ' ')
    return x

digits = [str(x) for x in range(10)] # Remove digits
digit_less = [full_remove(x, digits) for x in sentences]
punc_less = [full_remove(x, list(string.punctuation)) for x in
digit_less] # Remove punctuation
sents_lower = [x.lower() for x in punc_less] # Make everything lower-
case

#Stop words - Stop words are the words that are filtered out because
# they are believed to contain no useful information for the task at
hand.
# These usually include articles such as 'a' and 'the', pronouns such
# as 'i' and 'they', and prepositions such 'to' and 'from'.
# We have put together a very small list of stop words,
# but these are by no means comprehensive.

# Define our stop words
stop_set = set(['the', 'a', 'an', 'i', 'he', 'she', 'they', 'to',
'of', 'it', 'from'])
# Remove stop words
sents_split = [x.split() for x in sents_lower]
sents_processed = [" ".join(list(filter(lambda a: a not in stop_set,
x))) for x in sents_split]

```

```
#Let us look at the sentences:  
sents_processed[0:10]
```

Bag of words

- In order to use linear classifiers on our data set, we need to transform our textual data into numeric data. The classical way to do this is known as the bag of words representation.
- In this representation, each word is thought of as corresponding to a number in " $\{1, 2, \dots, V\}$ " where " V " is the size of our vocabulary. And each sentence is represented as a V -dimensional vector x , where x_i is the number of times that word i occurs in the sentence.
- To do this transformation, we will make use of the "CountVectorizer" class in "scikit-learn". We will cap the number of features at 4500, meaning a word will make it into our vocabulary only if it is one of the 4500 most common words in the corpus. This is often a useful step as it can weed out spelling mistakes and words which occur too infrequently to be useful.
- Finally, we will also append a '1' to the end of each vector to allow our linear classifier to learn a bias term.

```
# Transform to bag of words representation.  
vectorizer = CountVectorizer(analyzer = "word", tokenizer = None,  
preprocessor = None, stop_words = None, max_features = 4500)  
data_features = vectorizer.fit_transform(sents_processed)  
# Append '1' to the end of each vector.  
data_mat = data_features.toarray()  
  
#Training / test split - We split the data into a training set of 2500  
sentences  
# and a test set of 500 sentences (of which 250 are positive and 250  
negative).  
# Split the data into testing and training sets  
np.random.seed(0)  
test_inds = np.append(np.random.choice((np.where(y==-1))[0], 250,  
replace=False), np.random.choice((np.where(y==1))[0], 250,  
replace=False))  
train_inds = list(set(range(len(labels))) - set(test_inds))  
  
train_data = data_mat[train_inds,]  
train_labels = y[train_inds]  
  
test_data = data_mat[test_inds,]  
test_labels = y[test_inds]  
  
print("train data: ", train_data.shape)  
print("test data: ", test_data.shape)
```

2. Fitting a logistic regression model to the training data

- We could implement our own logistic regression solver using stochastic gradient descent, but fortunately, there is already one built into "scikit-learn".
- Due to the randomness in the SGD procedure, different runs can yield slightly different solutions (and thus different error values).

```
# Fit logistic classifier on training data
clf = SGDClassifier(loss="log", penalty="none")
clf.fit(train_data, train_labels)
# Pull out the parameters (w,b) of the logistic regression model
w = clf.coef_[0,:]
b = clf.intercept_
# Get predictions on training and test data
preds_train = clf.predict(train_data)
preds_test = clf.predict(test_data)
# Compute errors
errs_train = np.sum((preds_train > 0.0) != (train_labels > 0.0))
errs_test = np.sum((preds_test > 0.0) != (test_labels > 0.0))
print ("Training error: ", float(errs_train)/len(train_labels))
print ("Test error: ", float(errs_test)/len(test_labels))
```

3. Analyzing the margin

- The logistic regression model produces not just classifications but also conditional probability estimates.
- We will say that "x" has margin "gamma" if (according to the logistic regression model) $\Pr(y=1|x) > (1/2)+\text{gamma}$ or $\Pr(y=1|x) < (1/2)-\text{gamma}$. The following function margin_counts takes as input as the classifier ("clf", computed earlier), the test set ("test_data"), and a value of "gamma", and computes how many points in the test set have margin of at least "gamma".

```
# Return number of test points for which  $\Pr(y=1)$  lies in  $[0, 0.5 - \text{gamma})$  or  $(0.5 + \text{gamma}, 1]$ 
def margin_counts(clf, test_data, gamma):
    # Compute probability on each test point
    preds = clf.predict_proba(test_data)[:,1]
    # Find data points for which prediction is at least gamma away
    # from 0.5
    margin_inds = np.where((preds > (0.5+gamma)) | (preds < (0.5-
gamma))) [0]
    return float(len(margin_inds))

#Let us visualize the test set's distribution of margin values.
gammas = np.arange(0, 0.5, 0.01)
f = np.vectorize(lambda g: margin_counts(clf, test_data, g))
plt.plot(gammas, f(gammas) / 500.0, linewidth=2, color='green')
plt.xlabel('Margin', fontsize=14)
plt.ylabel('Fraction of points above margin', fontsize=14)
plt.show()
```

```

#We investigate a natural question: "Are points "x" with larger margin
more likely to
# be classified correctly? To address this, we define a function
margin_errors
# that computes the fraction of points with margin at least "gamma"
that are misclassified.
# Return error of predictions that lie in intervals [0, 0.5 - gamma)
and (0.5 + gamma, 1]
def margin_errors(clf, test_data, test_labels, gamma):
    # Compute probability on each test point
    preds = clf.predict_proba(test_data)[:,-1]
    # Find data points for which prediction is at least gamma away
    from 0.5
    margin_inds = np.where((preds > (0.5+gamma)) | (preds < (0.5-
gamma))) [0]
    # Compute error on those data points.
    num_errors = np.sum((preds[margin_inds] > 0.5) !=
(test_labels[margin_inds] > 0.0))
    return float(num_errors)/len(margin_inds)

#Let us visualize the relationship between margin and error rate.
# Create grid of gamma values
gammas = np.arange(0, 0.5, 0.01)
# Compute margin_errors on test data for each value of g
f = np.vectorize(lambda g: margin_errors(clf, test_data, test_labels,
g))
# Plot the result
plt.plot(gammas, f(gammas), linewidth=2)
plt.ylabel('Error rate', fontsize=14)
plt.xlabel('Margin', fontsize=14)
plt.show()

```

4. Words with large influence

- Finally, we attempt to partially interpret the logistic regression model.
- Which words are most important in deciding whether a sentence is positive? As a first approximation to this, we simply take the words whose coefficients in "w" have the largest positive values.
- Likewise, we look at the words whose coefficients in "w" have the most negative values, and we think of these as influential in negative predictions.

```

# Convert vocabulary into a list:
vocab = np.array([z[0] for z in sorted(vectorizer.vocabulary_.items(),
key=lambda x:x[1])])
# Get indices of sorting w
inds = np.argsort(w)
# Words with large negative values

```

```

neg_inds = inds[0:50]
print("Highly negative words: ")
print([str(x) for x in list(vocab[neg_inds])])
# Words with large positive values
pos_inds = inds[-49:-1]
print("Highly positive words: ")
print([str(x) for x in list(vocab[pos_inds])])

```

PROJECT 5: DAILY WEATHER DATA ANALYSIS USING DECISION TREE CLASSIFICATION

Classification of Weather Data using scikit-learn

We will use scikit-learn to perform a decision tree based classification of weather data.

```

#Project 5 : Daily Weather Data Analysis using Decision Tree
Classification
#Importing the Necessary Libraries
import pandas as pd
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

#Creating a Pandas DataFrame from a CSV file
data = pd.read_csv('data/data_weather.csv')
print("Columns are: ", data.columns)
print("Data: \n", data)
print("Null Data: \n", data[data.isnull().any(axis=1)])

```

Daily Weather Data Description

The file `daily_weather.csv` is a comma-separated file that contains weather data. This data comes from a weather station. The weather station is equipped with sensors that capture weather-related measurements such as air temperature, air pressure, and relative humidity. Data was collected for a period of three years, from September 2011 to September 2014, to ensure that sufficient data for different seasons and weather conditions is captured.

Let us now check all the columns in the data.

- Each row in `daily_weather.csv` captures weather data for a separate day.
- Sensor measurements from the weather station were captured at one-minute intervals. These measurements were then processed to generate values to describe daily weather. Since this dataset was created to classify low-humidity days vs. non-low-humidity days (that is, days with normal or high humidity), the variables included are weather measurements in the morning, with one measurement, namely relative humidity, in the afternoon. The idea is to use the morning weather

values to predict whether the day will be low-humidity or not based on the afternoon measurement of relative humidity.

- Each row, or sample, consists of the following variables:
 - number: unique number for each row
 - air_pressure_9am: air pressure averaged over a period from 8:55am to 9:04am (Unit: hectopascals)
 - air_temp_9am: air temperature averaged over a period from 8:55am to 9:04am (Unit: degrees Fahrenheit)
 - air_wind_direction_9am: wind direction averaged over a period from 8:55am to 9:04am (Unit: degrees, with 0 means coming from the North, and increasing clockwise)
 - air_wind_speed_9am: wind speed averaged over a period from 8:55am to 9:04am (Unit: miles per hour)
 - max_wind_direction_9am:** wind gust direction averaged over a period from 8:55am to 9:10am (Unit: degrees, with 0 being North and increasing clockwise*)
 - max_wind_speed_9am: wind gust speed averaged over a period from 8:55am to 9:04am (Unit: miles per hour)
 - rain_accumulation_9am: amount of rain accumulated in the 24 hours prior to 9am (Unit: millimeters)
 - rain_duration_9am: amount of time rain was recorded in the 24 hours prior to 9am (Unit: seconds)
 - relative_humidity_9am: relative humidity averaged over a period from 8:55am to 9:04am (Unit: percent)
 - relative_humidity_3pm: relative humidity averaged over a period from 2:55pm to 3:04pm (*Unit: percent *)

Data Cleaning Steps:

```
#Data Cleaning Steps
#We will not need the "number" column for each row so we can clean it.
del data['number']
#Let us drop null values using the pandas dropna function.
before_rows = data.shape[0]
print(before_rows)
data = data.dropna()
after_rows = data.shape[0]
print(after_rows)
#How many rows dropped due to cleaning?
print("Total rows dropped: ", before_rows - after_rows)
```

Convert to a Classification Task

```
#Convert to a Classification Task
```

```

#Binarize the relative_humidity_3pm to 0 or 1.
clean_data = data.copy()
clean_data['high_humidity_label'] =
(clean_data['relative_humidity_3pm'] > 24.99)*1
print(clean_data['high_humidity_label'])

#Target is stored in 'y'.
y=clean_data[['high_humidity_label']].copy()
clean_data['relative_humidity_3pm'].head()
print("Y Data: \n",y.head())

```

Use 9am Sensor Signals as Features to Predict Humidity at 3pm

```

#Use 9am Sensor Signals as Features to Predict Humidity at 3pm
morning_features =
['air_pressure_9am', 'air_temp_9am', 'avg_wind_direction_9am', 'avg_wind_
speed_9am',

'max_wind_direction_9am', 'max_wind_speed_9am', 'rain_accumulation_9am',
'rain_duration_9am']

X = clean_data[morning_features].copy()
print("Columns in X: ",X.columns)
print("Columns in Y: ",y.columns)

```

Perform Test and Train split

- In the training phase, the learning algorithm uses the training data to adjust the model's parameters to minimize errors. At the end of the training phase, we get the trained model.
- In the testing phase, the trained model is applied to test data. Test data is separate from the training data, and is previously unseen by the model. The model is then evaluated on how it performs on the test data. The goal in building a classifier model is to have the model perform well on training as well as test data.

```

#Perform Test and Train split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.33, random_state=324)
print ("X_train is as under:")
print(X_train.head())
print ("X_test is as under:")
print(X_test.head())
print ("y_train is as under:")
print(y_train.head())
print ("y_test is as under:")
print(y_test.head())

```

```
print ("Let us describe y_train")
y_train.describe()
#Fit on Train Set
humidity_classifier = DecisionTreeClassifier(max_leaf_nodes=10,
random_state=0)
humidity_classifier.fit(X_train, y_train)
type(humidity_classifier)
```

Predict on Test Set

```
#Predict on Test Set
predictions = humidity_classifier.predict(X_test)
print("Sample Predictions: \n",predictions[:10])
print("Sample Y Test (Actual Data): \n",y_test['high_humidity_label'][:10])

#Measure Accuracy of the Classifier
print("Accuracy: \n",accuracy_score(y_true = y_test, y_pred =
predictions))
```

PROJECT 6: NEAREST NEIGHBOUR FOR HANDWRITTEN DIGIT RECOGNITION

We will build a classifier that takes an image of a handwritten digit and outputs a label 0-9. We will look at a particularly simple strategy for this problem known as the nearest neighbor classifier.

To run this notebook we should have the following Python packages installed:

- numpy
- matplotlib
- sklearn`

1. The MNIST dataset

"MNIST" is a classic dataset in machine learning, consisting of 28x28 gray-scale images handwritten digits. The original training set contains 60,000 examples and the test set contains 10,000 examples. In this notebook we will be working with a subset of this data: a training set of 7500 examples and a test set of 1,000 examples.

About the dataset

Four files are available on this package:

- training set images
- training set labels
- test set images
- test set labels

These files are not in any standard image format. You have to write your own (very simple) program to read them. You can find more details about the dataset here: <http://yann.lecun.com/exdb/mnist/>

```
import numpy as np
import matplotlib.pyplot as plt
import time

#Load the training set:
train_data = np.load('data/train_data.npy')
train_labels = np.load('data/train_labels.npy')

#Load the testing set:
test_data = np.load('data/test_data.npy')
test_labels = np.load('data/test_labels.npy')

#Print out data dimensions:
print("Training dataset dimensions: ", np.shape(train_data))
print("Number of training labels: ", len(train_labels))
print("Testing dataset dimensions: ", np.shape(test_data))
print("Number of testing labels: ", len(test_labels))

#Compute the number of examples of each digit:
train_digits, train_counts = np.unique(train_labels,
return_counts=True)
print("Training set distribution:")
print(dict(zip(train_digits, train_counts)))
test_digits, test_counts = np.unique(test_labels, return_counts=True)
print("Test set distribution:")
print(dict(zip(test_digits, test_counts)))
```

2. Visualizing the data

```
#2. Visualizing the data
#Each data point is stored as 784-dimensional vector. To visualize a
data point,
# we first reshape it to a 28x28 image.
#Define a function that displays a digit given its vector
representation:
def show_digit(x):
    plt.axis('off')
    plt.imshow(x.reshape((28,28)), cmap=plt.cm.gray)
    plt.show()
    return

#Define a function that takes an index into a particular data set
("train" or "test") and displays that image.
def vis_image(index, dataset="train"):
```

```

if(dataset=="train"):
    show_digit(train_data[index,])
    label = train_labels[index]
else:
    show_digit(test_data[index,])
    label = test_labels[index]
print("Label " + str(label))
return
#View the first data point in the training set:
vis_image(0, "train")
#View the first data point in the test set:
vis_image(0, "test")

```

3. Squared Euclidean distance

- To compute nearest neighbors in our data set, we need to compute distances between data points. A natural distance function is *Euclidean distance*: for two vectors x, y , their Euclidean distance is defined as

$$\|x - y\| = \sqrt{\sum_{i=1}^d (x_i - y_i)^2}.$$

- Often we omit the square root, and simply compute *squared Euclidean distance*:

$$\|x - y\|^2 = \sum_{i=1}^d (x_i - y_i)^2.$$

- We will compute squared Euclidean distance. The following function does so:

```

#Function that computes squared Euclidean distance between two
vectors.
def squared_dist(x,y):
    return np.sum(np.square(x-y))
#Compute distance between a seven and a one in our training set.
vis_image(4, "train")
vis_image(5, "train")
print("Distance from 7 to 1: ",
squared_dist(train_data[4,],train_data[5,]))

#Compute distance between a seven and a two in our training set.
vis_image(4, "train")
vis_image(1, "train")
print("Distance from 7 to 2: ",
squared_dist(train_data[4,],train_data[1,]))

#Compute distance between two seven's in our training set.
vis_image(4, "train")
vis_image(7, "train")
print("Distance from 7 to 7: ",
squared_dist(train_data[4,],train_data[7,]))

```

4. Computing nearest neighbors

Now that we have a distance function defined, we can now turn to nearest neighbor classification.

Function that takes a vector x and returns the index of its nearest neighbor in `train_data`.

```
#4. Computing nearest neighbors
#Now that we have a distance function defined,
# we can now turn to nearest neighbor classification.

#Function that takes a vector x and returns the index
# of its nearest neighbor in train_data.
def find_NN(x):
    # Compute distances from x to every row in train_data
    distances = [squared_dist(x, train_data[i,]) for i in
range(len(train_labels))]
    # Get the index of the smallest distance
    return np.argmin(distances)

#Function that takes a vector x and returns the class of its nearest
neighbor in train_data.
def NN_classifier(x):
    # Get the index of the the nearest neighbor
    index = find_NN(x)
    # Return its class
    return train_labels[index]

#A success case:
print("A success case:")
print("NN classification: ", NN_classifier(test_data[0,]))
print("True label: ", test_labels[0])
print("The test image:")
vis_image(0, "test")
print("The corresponding nearest neighbor image:")
vis_image(find_NN(test_data[0,]), "train")

#A failure case:
print("A failure case:")
print("NN classification: ", NN_classifier(test_data[39,]))
print("True label: ", test_labels[39])
print("The test image:")
vis_image(39, "test")
print("The corresponding nearest neighbor image:")
vis_image(find_NN(test_data[39,]), "train")
```

5. Processing the full test set

- Let us apply our nearest neighbor classifier over the full data set.
- To classify each test point, our code takes a full pass over each of the 7500 training examples. Thus we should not expect testing to be very fast.

- Let us predict on each test data point (and time it!)

```
#5. Processing the full test set
t_before = time.time()
test_predictions = [NN_classifier(test_data[i,]) for i in
range(len(test_labels))]
t_after = time.time()
#Compute the error:
err_positions = np.not_equal(test_predictions, test_labels)
error = float(np.sum(err_positions))/len(test_labels)
print("Error of nearest neighbor classifier: ", error)
print("Classification time (seconds): ", t_after - t_before)
```

6. Faster nearest neighbor methods

- Performing nearest neighbor classification in the way we have presented requires a full pass through the training set in order to classify a single point. If there are N training points in $\mathbb{R}^d$, this takes $O(Nd)$ time.
- Fortunately, there are faster methods to perform nearest neighbor. "scikit-learn" has fast implementations of two useful nearest neighbor data structures: the ball tree and the k-d tree.

```
#6. Faster nearest neighbor methods
from sklearn.neighbors import BallTree
# Build nearest neighbor structure on training data
t_before = time.time()
ball_tree = BallTree(train_data)
t_after = time.time()
# Compute training time
t_training = t_after - t_before
print("Time to build data structure (seconds): ", t_training)
# Get nearest neighbor predictions on testing data
t_before = time.time()
test_neighbors = np.squeeze(ball_tree.query(test_data, k=1,
return_distance=False))
ball_tree_predictions = train_labels[test_neighbors]
t_after = time.time()
# Compute testing time
t_testing = t_after - t_before
print("Time to classify test set (seconds): ", t_testing)
# Verify that the predictions are the same
print("Ball tree produces same predictions as above? ",
np.array_equal(test_predictions, ball_tree_predictions))

from sklearn.neighbors import KDTree
# Build nearest neighbor structure on training data
t_before = time.time()
kd_tree = KDTree(train_data)
t_after = time.time()
# Compute training time
```

```

t_training = t_after - t_before
print("Time to build data structure (seconds): ", t_training)
# Get nearest neighbor predictions on testing data
t_before = time.time()
test_neighbors = np.squeeze(kd_tree.query(test_data, k=1,
return_distance=False))
kd_tree_predictions = train_labels[test_neighbors]
t_after = time.time()
# Compute testing time
t_testing = t_after - t_before
print("Time to classify test set (seconds): ", t_testing)
# Verify that the predictions are the same
print("KD tree produces same predictions as above? ",
np.array_equal(test_predictions, kd_tree_predictions))

```

PROJECT 7:

- We will learn how to perform k-means clustering using scikit-learn in Python.
- We will use cluster analysis to generate a big picture model of the weather at a local station using a minute-granularity data. In this dataset, we have in the order of millions records. How do we create 12 clusters out of them?

```

#Importing the Necessary Libraries
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import pandas as pd
import numpy as np
from itertools import cycle, islice
import matplotlib.pyplot as plt
from pandas.plotting import parallel_coordinates
#Creating a Pandas DataFrame from a CSV file
data = pd.read_csv('data/weather.zip',compression = 'zip')

print("Data Shape: ",data.shape)
print("Sample Data: \n",data.head())

```

Minute Weather Data Description

- The minute weather dataset comes from the same source as the daily weather dataset that we used in the decision tree based classifier notebook. The main difference between these two datasets is that the minute weather dataset contains raw sensor measurements captured at one-minute intervals. Daily weather dataset instead contained processed and well curated data. The data is in the file minute_weather.csv, which is a comma-separated file.

- As with the daily weather data, this data comes from a weather station. The weather station is equipped with sensors that capture weather-related measurements such as air temperature, air pressure, and relative humidity. Data was collected for a period of three years, from September 2011 to September 2014, to ensure that sufficient data for different seasons and weather conditions is captured.
- Each row in `minute_weather.csv` contains weather data captured for a one-minute interval. Each row, or sample, consists of the following variables:
 - `rowID`: unique number for each row (Unit: NA)
 - `hpwren_timestamp`: timestamp of measure (Unit: year-month-day hour:minute:second)
 - `air_pressure`: air pressure measured at the timestamp (Unit: hectopascals)
 - `air_temp`: air temperature measure at the timestamp (Unit: degrees Fahrenheit)
 - `avg_wind_direction`: wind direction averaged over the minute before the timestamp (Unit: degrees, with 0 means coming from the North, and increasing clockwise)
 - `avg_wind_speed`: wind speed averaged over the minute before the timestamp (Unit: meters per second)
 - `max_wind_direction`: highest wind direction in the minute before the timestamp (Unit: degrees, with 0 being North and increasing clockwise)
 - `max_wind_speed`: highest wind speed in the minute before the timestamp (Unit: meters per second)
 - `min_wind_direction`: smallest wind direction in the minute before the timestamp (Unit: degrees, with 0 being North and increasing clockwise)
 - `min_wind_speed`: smallest wind speed in the minute before the timestamp (Unit: meters per second)
 - `rain_accumulation`: amount of accumulated rain measured at the timestamp (Unit: millimeters)
 - `rain_duration`: length of time rain has fallen as measured at the timestamp (Unit: seconds)
 - `relative_humidity`: relative humidity measured at the timestamp (Unit: percent)

```
###Data Sampling
#Lots of rows, so let us sample down by taking every 10th row.
sampled_df = data[(data['rowID'] % 10) == 0]
print("Sample Data Shape: ",sampled_df.shape)

#Statistics
sampled_df.describe().transpose()
sampled_df[sampled_df['rain_accumulation'] == 0].shape
sampled_df[sampled_df['rain_duration'] == 0].shape

#Drop all the Rows with Empty rain duration and rain_accumulation
del sampled_df['rain_accumulation']
del sampled_df['rain_duration']
rows_before = sampled_df.shape[0]
sampled_df = sampled_df.dropna()
rows_after = sampled_df.shape[0]

#How many rows did we drop ?
```

```

print("How many rows did we drop ? ",rows_before - rows_after)
print("Columns in Sample Data: ",sampled_df.columns)

#Select Features of Interest for Clustering
features = ['air_pressure', 'air_temp', 'avg_wind_direction',
'avg_wind_speed', 'max_wind_direction',
            'max_wind_speed','relative_humidity']
select_df = sampled_df[features]
select_df.columns
select_df

#Scale the Features using StandardScaler
X = StandardScaler().fit_transform(select_df)
X

#Use k-Means Clustering
kmeans = KMeans(n_clusters=12)
model = kmeans.fit(X)
print("model\n", model)

#What are the centers of 12 clusters we formed ?
centers = model.cluster_centers_
print("centers: ", centers)
###Plots
#Let us first create some utility functions which will help us in
plotting graphs:
# Function that creates a DataFrame with a column for Cluster Number

def pd_centers(featuresUsed, centers):
    colNames = list(featuresUsed)
    colNames.append('prediction')

    # Zip with a column called 'prediction' (index)
    Z = [np.append(A, index) for index, A in enumerate(centers)]

    # Convert to pandas data frame for plotting
    P = pd.DataFrame(Z, columns=colNames)
    P['prediction'] = P['prediction'].astype(int)
    return P

# Function that creates Parallel Plots

def parallel_plot(data):
    my_colors = list(islice(cycle(['b', 'r', 'g', 'y', 'k']), None,
len(data)))
    plt.figure(figsize=(15,8)).gca().axes.set_ylim([-3,+3])
    parallel_coordinates(data, 'prediction', color = my_colors,
marker='o')
    plt.show()

P = pd_centers(features, centers)

```

```
print("pd_centers: ",P)
```

Output

```
###Dry Days
parallel_plot(P[P['relative_humidity'] < -0.5])

###Warm Days
parallel_plot(P[P['air_temp'] > 0.5])

###Cool Days
parallel_plot(P[(P['relative_humidity'] > 0.5) & (P['air_temp'] <
0.5)])
```

PROJECT 8:

We will be predicting the sentiment (positive or negative) of a single sentence taken from a review of a movie, restaurant, or product. The data set consists of 3000 labeled sentences, which we divide into a training set of size 2500 and a test set of size 500. We have already used a logistic regression classifier. Now, we will use a support vector machine.

```
import string
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
matplotlib.rc('xtick', labels=14)
matplotlib.rc('ytick', labels=14)
from sklearn.feature_extraction.text import CountVectorizer
from sklearn import svm

#Loading and preprocessing the data
#Read in the data set:
with open("data/full_set.txt") as f:
    content = f.readlines()
#Remove leading and trailing white space:
content = [x.strip() for x in content]

#Separate the sentences from the labels:
sentences = [x.split("\t")[0] for x in content]
labels = [x.split("\t")[1] for x in content]

#Transform the labels from '0 versus 1' to '-1 versus 1':
```

```

y = np.array(labels, dtype='int8')
y = 2*y - 1

#Let us define a function, "full_remove" that takes a string x
# and a list of characters from a removal_list and returns with all
the
# characters in removal_list replaced by ' '.
def full_remove(x, removal_list):
    for w in removal_list:
        x = x.replace(w, ' ')
    return x
#Remove digits:
digits = [str(x) for x in range(10)]
digit_less = [full_remove(x, digits) for x in sentences]
#Remove punctuation:
punc_less = [full_remove(x, list(string.punctuation)) for x in
digit_less]
#Make everything lower-case:
sents_lower = [x.lower() for x in punc_less]
#Define our stop words:
stop_set = set(['the', 'a', 'an', 'i', 'he', 'she', 'they', 'to',
'of', 'it', 'from'])
#Remove stop words
sents_split = [x.split() for x in sents_lower]
sents_processed = [" ".join(list(filter(lambda a: a not in stop_set,
x))) for x in sents_split]

#Transform to bag of words representation:
vectorizer = CountVectorizer(analyzer = "word", tokenizer = None,
preprocessor = None, stop_words = None, max_features = 4500)
data_features = vectorizer.fit_transform(sents_processed)

#Append '1' to the end of each vector.
data_mat = data_features.toarray()

#Split the data into testing and training sets:
np.random.seed(0)
test_inds = np.append(np.random.choice((np.where(y==-1))[0], 250,
replace=False), np.random.choice((np.where(y==1))[0], 250,
replace=False))
train_inds = list(set(range(len(labels)) - set(test_inds))
train_data = data_mat[train_inds,]
train_labels = y[train_inds]
test_data = data_mat[test_inds,]
test_labels = y[test_inds]
print("train data: ", train_data.shape)
print("test data: ", test_data.shape)

```

Fitting a support vector machine to the data

In support vector machines, we are given a set of examples $(x_1, y_1), \dots, (x_n, y_n)$ and we want to find a weight vector $w \in \mathbb{R}^d$ that solves the following optimization problem:

$$\min_{w \in \mathbb{R}^d} \|w\|^2 + C \sum_{i=1}^n \xi_i$$

subject to $y_i \langle w, x_i \rangle \geq 1 - \xi_i$ for all $i = 1, \dots, n$

scikit-learn provides an SVM solver that we will use. The following routine takes the constant "C" as an input (from the above optimization problem) and returns the training and test error of the resulting SVM model. It is invoked as follows:

- "training_error, test_error = fit_classifier(C)"
- The default value for parameter "C" is 1.0.

```
#Fitting a support vector machine to the data
def fit_classifier(C_value=1.0):
    clf = svm.LinearSVC(C=C_value, loss='hinge')
    clf.fit(train_data, train_labels)
    # Get predictions on training data
    train_preds = clf.predict(train_data)
    train_error = float(np.sum((train_preds > 0.0) != (train_labels >
0.0)))/len(train_labels)
    # Get predictions on test data
    test_preds = clf.predict(test_data)
    test_error = float(np.sum((test_preds > 0.0) != (test_labels >
0.0)))/len(test_labels)
    return train_error, test_error

cvals = [0.01, 0.1, 1.0, 10.0, 100.0, 1000.0, 10000.0]
for c in cvals:
    train_error, test_error = fit_classifier(c)
    print ("Error rate for C = %0.2f: train %0.3f test %0.3f" % (c,
train_error, test_error))

#We see that the minimum test error is for C = 1.0.
```

We see that the minimum test error is for C = 1.0.

3. Evaluating C by k-fold cross-validation

- As we can see, the choice of "C" has a very significant effect on the performance of the SVM classifier. We were able to assess this because we have a separate test set. In general, however, this is a luxury we won't possess.
- A reasonable way to estimate the error associated with a specific value of "C" is by k-fold cross validation:
 - Partition the training set into "k" equal-sized subsets.
 - For $i=1, 2, \dots, k$, train a classifier with parameter C..

- Average the errors: $(e_1 + \dots + e_k)/k$
- The following procedure, `cross_validation_error`, does exactly this. It takes as input:
 - the training set "x,y"
 - the value of "C" to be evaluated
 - the integer "k"
- It returns the estimated error of the classifier for that particular setting of "C".

```
#3. Evaluating C by k-fold cross-validation
def cross_validation_error(x,y,C_value,k):
    n = len(y)
    # Randomly shuffle indices
    indices = np.random.permutation(n)
    # Initialize error
    err = 0.0
    # Iterate over partitions
    for i in range(k):
        # Partition indices
        test_indices = indices[int(i*(n/k)):int((i+1)*(n/k) - 1)]
        train_indices = np.setdiff1d(indices, test_indices)
        # Train classifier with parameter c
        clf = svm.LinearSVC(C=C_value, loss='hinge')
        clf.fit(x[train_indices], y[train_indices])
        # Get predictions on test partition
        preds = clf.predict(x[test_indices])
        # Compute error
        err += float(np.sum((preds > 0.0) != (y[test_indices] >
0.0)))/len(test_indices)
    return err/k

#Let us print out the errors or differences of k
for k in range(2,10):
    print("cross_validation_error: ")
    print(cross_validation_error(train_data,train_labels,1.0,k))
```